

Product Requirements Document (PRD)

StockSentinel - SME Inventory Tracking & Alert System

Document version	1.0.0
Status	Ready for Review
Author	Product Management
Date	May 23, 2026

This version reorganizes the original PRD into a clearer structure for review and implementation planning.

1. Executive Summary

Product summary. StockSentinel is a mobile-first inventory tracking system for small and medium-sized enterprises (SMEs). It replaces paper ledgers and complex ERP tools with a lightweight digital workflow that records stock movements, calculates current inventory dynamically, and alerts owners before items run out.

Problem. Many retail and hospitality SMEs do not have real-time stock visibility. As a result, they suffer from stockouts, lost sales, rushed purchasing, excess stock, and wasted cash flow.

Vision. Give small business owners the same level of inventory control, data accuracy, and replenishment insight that larger retailers have, without requiring heavy setup or technical overhead.

2. Business Objectives and Success Metrics

Objective	Success metric	Target	Timeframe
Reduce stockouts	Unexpected out-of-stock incidents	< 2 per month	Within 3 months
Optimize working capital	Inventory carrying cost reduction	15% - 25% reduction	Within 6 months
Streamline operations	Pick list accuracy	> 98% error-free picking	Within 3 months
Increase retention	Weekly active users	80% of registered staff active	Continuous
Maintain reliability	API/database sync latency	< 2 seconds	Continuous

3. Target Users

The system is designed for two core roles with role-based access control:

Role	Main needs	Main pain points
Shop owner / Admin	Live inventory values, low-stock alerts, purchase order support, dashboards	Too much cash tied in stock, manual reorder calculations, late discovery of stockouts
Floor / warehouse worker	Fast mobile scanning, check-in/check-out, bin locations	Spreadsheet errors, complex software, disorganized storerooms

4. Core Product Features

Product catalog and supplier directory.

- A master SKU list with product name, cost, retail price, category, and bin location.
- A supplier directory with contact details, payment terms, and lead-time history.

Transaction-based inventory ledger.

- Inventory is recorded as stock movement transactions instead of a static on-hand number.
- Each transaction stores date, item, location, user, type, quantity, and reference number.

Low-stock alert engine.

- Reorder points are calculated from sales history and supplier lead time.
- Items below the reorder point are highlighted in light red on the dashboard.

Mobile barcode and QR scanning.

- Staff can scan barcodes or QR codes using a phone camera in mobile browsers.

- Scan results show stock status, bin location, cost, and inventory value.

Automated alerts.

- Low-stock email digests summarize items that need replenishment.
- Critical stock SMS alerts notify the owner when a key item reaches zero.

Reorder-point formula: $ROP = (ADUS \times LT) + SS$

Safety stock formula: $SS = (MDS \times MLT) - (ADUS \times ALT)$

5. Data Model and Inventory Logic

StockSentinel should use a relational database such as PostgreSQL or Airtable so that every movement is tied to a product, location, and user.

Table	Purpose	Key fields
Suppliers	Stores vendor information	SupplierID, SupplierName, ContactName, Email, PhoneNumber, PaymentTerms
Products	Stores item master data	ProductID, SKU, ProductName, Category, UnitCost, RetailPrice, ReorderThreshold, SupplierID, IsActive
Locations	Stores bin or room locations	LocationID, LocationName, LocationType, IsActive
Users	Stores staff and access roles	UserID, FullName, Email, Role
Transactions	Stores every stock movement	TransactionID, TransactionDate, ProductID, LocationID, UserID, TransactionType, Quantity, ReferenceNumber, Notes

Inventory should not be stored as a manually edited 'quantity on hand' field. Instead, current stock is calculated from transactions:

$Inventory = \text{sum(Inbound)} - \text{sum(Outbound)} +/- \text{sum(Adjustment)}$

$Valuation = \text{sum(Inventory per product} \times \text{UnitCost per product)}$

6. Platform and Integrations

Option	Description	Best fit
Glide Tables	Native low-code setup with role-based screens and mobile scanning	Fastest path for a lightweight MVP
Airtable + Softr	Airtable stores data and Softr provides the front-end portal	Good for a polished client portal
Google Sheets + Apps Script	Automated scripts read data and send alerts	Lowest-cost option, but less robust

Key integrations:

- Twilio via Zapier for SMS notifications using E.164 phone numbers.
- SMTP service or Gmail API for scheduled low-stock email digests.

7. User Experience Guidelines

- Design for mobile first so staff can scan and update inventory on the store floor.
- Highlight low-stock items in light red and healthy stock levels in green.
- Keep the transaction flow short: Scan -> enter quantity -> submit.
- Support search and filtering by SKU, category, and location.

8. Assumptions, Constraints, and Dependencies

Type	Summary
Assumption	Users have stable Wi-Fi or cellular internet at the store or warehouse.
Constraint	The solution should stay low cost and avoid heavy ERP integrations.
Dependency	Camera access must be granted for scanning; SMS delivery depends on valid Twilio credits and working webhooks.

End of document